WEEK 10

# Basic MQL and Document Modeling

Query nested documents safely, observe write results, and decide what belongs together from access patterns.

10

# SQL concepts provide a bridge, not a one-to-one translation

| Question part | PostgreSQL example | MongoDB example | Semantic caution |
| --- | --- | --- | --- |
| Source | FROM tickets | db.tickets | database and collection context |
| Filter | WHERE status = 'open' | { status: 'open' } | document and BSON type match |
| Projection | SELECT ticket_id | { ticket_id: 1, _id: 0 } | inclusion/exclusion rules |
| Order + limit | ORDER BY ... LIMIT | .sort(...).limit(...) | cursor operation order |

# Connect to Atlas without weakening the credential boundary

1. Copy the current mongodb+srv URI for the intended free deployment and database user.
2. Enter the URI at runtime with getpass; never print or commit it.
3. Permit only the temporary network path needed by the runtime and run ping first.
4. If TLS fails, inspect URI, driver, DNS, clock, and network; do not use tlsInsecure=True.

# A filter selects documents; a projection selects returned fields

```
filter_doc = {
  "status": {"$in": ["new", "open", "in_progress"]},
  "priority": "high"
}
projection = {
  "_id": 0, "ticket_id": 1,
  "status": 1, "subject": 1
}

for doc in tickets.find(filter_doc, projection) \
+                  .sort("ticket_id", 1):
    print(doc)
```

## Predict

- One output item per matching document
- Only named fields plus the chosen _id behavior
- BSON field types must match the filter values

# Nested paths and arrays require document-aware predicates

```
tickets.find({
  "requester.neighborhood": "Harbor",
  "events": {
    "$elemMatch": {
      "type": "status_changed",
      "to": "resolved"
    }
  }
})
```

## Why elemMatch?

- Dot notation reaches a nested field.
- The array contains event objects.
- Both event conditions must hold in the same element.

# Write results separate finding a target from changing its value

## matched_count answers ‘found’; modified_count answers ‘changed.’

### MATCHED

Documents satisfying the update filter.

### MODIFIED

Matched documents whose stored values actually changed.

### VERIFY

A follow-up read confirms the intended field and no broader scope.

# A safe delete uses a narrow filter and verified result

## Unsafe pattern

- Delete with an empty or broad filter
- Assume deleted_count without previewing identity
- Test in a shared collection with no reset

## Controlled pattern

- Preview exact identifiers with the intended filter
- Delete one disposable document and inspect deleted_count
- Verify absence and restore/reset the fixture

# Lab 1: Basic MQL with observable write evidence

Query a safe ticket collection, update one known document, verify it, and record one connection control.

- Run filters and projections for scalar, nested, and array fields; verify identifiers.
- Update one known ticket and record matched, modified, and follow-up-read evidence.
- Use the controlled delete/reset step and record how temporary Atlas network access was narrowed.

**SUBMIT ONE THING / one completed MQL notebook evidence record**

DAY 2

# Model around access, update ownership, and growth

What belongs together depends on how the system reads, changes, and bounds the aggregate.

# Embedding and referencing move work to different operations

## Embed when the aggregate is bounded

- Related facts are normally read and changed together
- Child data belongs to one parent and stays reasonably small
- One-document atomicity and locality are valuable

## Reference when identity is independent

- Related data is shared, unbounded, or changes independently
- One source of truth must serve many parents
- Separate lifecycle and targeted updates matter

# Unbounded arrays turn convenience into growth risk

## ‘Read together’ is not enough; ask how many and for how long.

### CARDINALITY

One, few, many, or effectively unbounded related items.

### GROWTH

Arrival rate, retention, document limit, and index amplification.

### LIFECYCLE

Who archives, deletes, repairs, or reads historical items independently?

# A model decision follows five workload questions

Write the questions before drawing the final document.

1. **Read**
   Which fields and relationships are retrieved together?
2. **Write**
   Which facts change together and how often?
3. **Own**
   Where is the authoritative identity or snapshot meaning?
4. **Bound**
   How many related items can exist over time?
5. **Verify**
   Which query and update prove the design serves the workload?

# Live demonstration and individual practice stay distinct

| Role | MongoDB University activity | Course purpose | Evidence |
| --- | --- | --- | --- |
| Instructor live | Modeling Data Relationships | show one embedded and one referenced example | whole-class reasoning |
| Student individual | Relational (SQL) to Document Model | translate a small relational design and compare shapes | completion plus model decision |
| Open fallback | course module + notebook fixture | preserve the same access-pattern reasoning | course-authored response |

# Lab 2: Defend one document boundary

Complete the distinct modeling activity, then justify embedding, referencing, or a hybrid for one relationship.

- Complete Relational (SQL) to Document Model individually or use the open fallback.
- Draw or write one candidate document shape for the assigned Metro Support access pattern.
- Defend it with one read, one update, ownership, boundedness, and one tradeoff.

**SUBMIT ONE THING / one model decision with activity evidence**

# MQL operates the document; access patterns justify its shape

**Query nested data precisely, interpret write evidence, and choose boundaries from lifecycle rather than fashion.**

- Which document identifiers should the filter return?
- What do matched and modified counts prove?
- Which read, update, and growth facts justify embedding or referencing?